HONEY-LLM: AN INTERACTIVE, SELF-HEALING HONEYPOT DEFENSE ECOSYSTEM FOR AGENTIC AI

Capstone Project Report

MID SEMESTER EVALUATION (Phases 1 to 4 Progress)

Submitted by:

(102303312) ANOUSHKA SINGH
(102303315) TARUN KRISHNA SHASTRI
(102303631) DEVANSH WADHWANI
(102303684) SHREYA GIRI

BE Third Year, Computer Engineering (CoE)
CPG No: 75

Under the Mentorship of:

Dr. Saif Nalband
Assistant Professor, Computer Science and Engineering Department

Computer Science and Engineering Department
Thapar Institute of Engineering and Technology, Patiala

August 2026

ABSTRACT

As generative Artificial Intelligence and Large Language Models (LLMs) transition from exploratory conversational tools to autonomous enterprise agents capable of executing multi-turn workflows, they introduce critical security vulnerabilities. Chief among these is adversarial prompt injection, where attackers manipulate natural-language instructions to bypass safety guardrails, hijack system roles, and exfiltrate proprietary corporate assets. Conventional perimeter defenses, including static Web Application Firewalls (WAFs) and rigid keyword matchers, operate on a reactive rejection paradigm that inadvertently reveals filter boundaries to attackers while failing against multi-turn semantic chaining.

This capstone project presents the design, system architecture, and verified implementation of **Honey-LLM**, covering work completed across **Phases 1 through 4** of the academic project roadmap. Specifically, the mid-semester implementation achieves four core deliverables: (1) an 8-class Adversarial Threat Taxonomy tailored to conversational enterprise agents; (2) a multi-tier *Intent Sieve* combining a sub-millisecond Tier-1 statistical classifier with an authoritative 8B moderation model governed by a custom prompt injection policy (Llama-Guard 3 [11]), achieving a **98.3% detection rate** (559/569 adversarial payloads intercepted) while maintaining a **0.0% False Positive Rate** on the benign evaluation set (0/320 legitimate customer queries flagged); (3) a containerized zero-trust deception sandbox termed the *Mirror Maze* running an LLM-driven decoy persona that dynamic-hallucinates synthetic bait to absorb attacker reconnaissance (verified **5/5 on container isolation tests**); and (4) an *Autonomous Guardrail Synthesis* feedback loop that distills captured exploit patterns into formal **NVIDIA NeMo Colang** rules [6], hot-patching live gateway policies in **10.4 seconds** with zero service interruption.

The subsequent project lifecycle, comprising Phase 5 (Forensic Telemetry and Live SOC Threat Intelligence Dashboard visualization) and Phase 6 (Empirical Red-Teaming at scale via multi-converter Microsoft PyRIT campaigns [13] and concurrency load profiling), is established as the structured roadmap for the final semester evaluation.

Keywords: Generative AI Security, Prompt Injection, Semantic Intent Sieve, LLM Honeypot, Autonomous Guardrails, NVIDIA NeMo, Zero-Trust Containerization.

DECLARATION

We hereby declare that the design principles, experimental methodologies, system implementation, and working prototype model of the capstone project entitled **"HONEY-LLM: AN INTERACTIVE, SELF-HEALING HONEYPOT DEFENSE ECOSYSTEM FOR AGENTIC AI"** is an authentic record of our own work completed up to **Phase 4 (Autonomous Guardrail Synthesis and Policy Hardening)** in the Computer Science and Engineering Department, Thapar Institute of Engineering and Technology (TIET), Patiala, under the mentorship and guidance of **Dr. Saif Nalband** during the academic semester (August 2026).

We further confirm that this report has not been submitted in part or full to any other University or Institution for the award of any degree or diploma.

Date: August 20, 2026

Roll No.	Name of Student	Signature
102303312	Anoushka Singh	_____
102303315	Tarun Krishna Shastri	_____
102303631	Devansh Wadhwani	_____
102303684	Shreya Giri	_____

Counter Signed By:

Faculty Mentor:

Dr. Saif Nalband
Assistant Professor, CSED
TIET, Patiala

Head of Department:

Dr. Neeraj Kumar
Professor & Head, CSED
TIET, Patiala

ACKNOWLEDGEMENT

We would like to express our deepest gratitude and heartfelt thanks to our respected project mentor, **Dr. Saif Nalband**, Assistant Professor, Computer Science and Engineering Department, Thapar Institute of Engineering and Technology, Patiala. His profound domain expertise, constructive technical criticism, constant encouragement, and intellectual guidance throughout the formulation and implementation of the initial four phases of **Honey-LLM** have been indispensable in steering this research to a successful milestone.

We extend our sincere thanks to **Dr. Neeraj Kumar**, Professor and Head of the Computer Science and Engineering Department, for providing state-of-the-art laboratory infrastructure, computational facilities, and an academic environment conducive to advanced systems research.

We also acknowledge the collective support of the faculty and technical staff of the Computer Science and Engineering Department at TIET, whose valuable academic perspectives helped refine our software architecture and evaluation methodologies. Furthermore, we are deeply grateful to our peers who supported adversarial dataset curation.

Lastly, we express our profound gratitude to our families and parents for their unyielding patience, emotional encouragement, and steadfast moral support throughout our academic journey.

Project Team Members:

Anoushka Singh (102303312), Tarun Krishna Shastri (102303315),
Devansh Wadhwani (102303631), Shreya Giri (102303684)

TABLE OF CONTENTS

TABLE OF CONTENTS (Continued)

LIST OF TABLES

LIST OF FIGURES

LIST OF ABBREVIATIONS

Abbreviation	Full Expansion / Description
AI	Artificial Intelligence
LLM	Large Language Model
SLM	Small Language Model
RAG	Retrieval-Augmented Generation
WAF	Web Application Firewall
SOC	Security Operations Center
FPR	False Positive Rate
FNR	False Negative Rate
OWASP	Open Worldwide Application Security Project
NeMo	Neural Modules (NVIDIA Conversational AI & Guardrails Framework)
TF-IDF	Term Frequency – Inverse Document Frequency
API	Application Programming Interface
REST	Representational State Transfer
JSONL	JavaScript Object Notation Lines
PyRIT	Python Risk Identification Tool for Generative AI (Microsoft)
SRS	Software Requirement Specification
WBS	Work Breakdown Structure
CSED	Computer Science and Engineering Department
TIET	Thapar Institute of Engineering and Technology

CHAPTER 1: INTRODUCTION

1.1 Project Overview

In the contemporary enterprise computing landscape of 2026, Large Language Models (LLMs) have evolved beyond isolated text generation interfaces into deeply integrated autonomous agents. Modern enterprise deployments rely on LLMs to automate mission-critical customer operations, query private structured databases, orchestrate multi-step API workflows, and execute tool-use tasks [7]. However, this rapid operational adoption has outpaced conventional cybersecurity paradigms, exposing a profound vulnerability surface known as the semantic attack vector [9].

Unlike traditional software systems where security boundaries are strictly demarcated between binary executable code and passive data buffers, LLMs process system instructions, operational context, and untrusted user inputs within a single unified semantic channel. Consequently, malicious actors exploit this architectural reality through **Adversarial Prompt Injection** and **Jailbreaking** techniques [9]. Attackers craft persuasive, contextually masked natural-language payloads, ranging from direct role overrides (such as instructing the model to ignore prior directives and output credentials) to indirect prompt injections embedded in retrieved enterprise documents, to manipulate the underlying model into bypassing access controls.

Traditional perimeter defenses, such as Web Application Firewalls (WAFs), heuristic keyword matchers, and static regular expressions, are fundamentally inadequate against semantic attacks. They lack linguistic context, cannot track conversational state across multi-turn sessions, and are trivially bypassed through character obfuscation, multilingual encoding, or subtle adversarial paraphrasing. More critically, standard security mechanisms follow a rigid block-and-alert model. When a malicious query is blocked with an explicit refusal message, the attacker immediately learns the perimeter filtering boundary and iterates their attack prompt until an evasion succeeds.

To decisively overcome these defensive limitations, this capstone project develops and demonstrates **Honey-LLM**: an interactive, self-hardening defense ecosystem for conversational AI architectures. Rather than simply rejecting malicious probes, Honey-LLM operates on a proactive deception philosophy. For the **Mid-Semester Evaluation**, the project team has fully developed, integrated, and verified the first four engineering phases:

- **Phase 1 (Adversarial Profiling & Threat Taxonomy):** Formulated an 8-class threat taxonomy mapping prompt injections to specific enterprise manifestations and validated concurrent dual-model local inference on Apple Silicon hardware.
- **Phase 2 (The Multi-Tier Semantic Intent Sieve):** Constructed an intelligent input-filtering pipeline that inspects queries in real time, pairing a sub-millisecond Tier-1 statistical classifier with an authoritative 8B moderation model governed by a custom prompt injection policy (Llama-Guard 3 [11]), achieving a 98.3% detection rate (559/569 adversarial payloads intercepted) at a 0.0% False Positive Rate on benign domain traffic (0/320 benign queries flagged).
- **Phase 3 (The 'Mirror Maze' Deception Honeypot):** Deployed an isolated zero-trust Docker sandbox hosting the 'Sarah' decoy persona, which dynamic-hallucinates synthetic bait to absorb attacker reconnaissance without leaking real infrastructure.
- **Phase 4 (Autonomous Guardrail Synthesis):** Implemented a closed self-healing loop that distills captured exploits into validated NVIDIA NeMo Colang rules [6], hot-patching live gateway policies in 10.4 seconds with zero downtime.

1.2 Need Analysis

The imperative for Honey-LLM is substantiated by critical operational realities observed across enterprise AI deployments in 2026:

1.2.1 The 'Smart Mirror' Trap: Enterprise Adoption vs. Defensive Lag

Industry surveys indicate that enterprise adoption of conversational AI agents has expanded rapidly across customer-facing workflows [7]. However, defensive capabilities have lagged behind offensive prompt exploitation techniques. Conventional static honeypots are quickly identified and abandoned by automated scanners. In contrast, generative honeypots offer dynamic semantic interaction, creating an essential observation window to capture zero-day exploitation techniques before they reach production services.

1.2.2 The Shift: Machine-Speed Autonomous Warfare

With conversational models managing automated customer dialogues [7], adversarial techniques have shifted from manual, one-off jailbreaks to automated, machine-speed offensive frameworks (such as ARACNE, Garak, and Microsoft PyRIT [13]). Automated offensive agents can systematically discover exploitable prompt sequences across iterative conversational turns, compressing vulnerability discovery timelines and rendering human-reliant triage workflows ineffective.

1.2.3 The 'Shadow Trust' Gap: Vulnerability of the Semantic Layer

Prompt injection is categorized as the primary vulnerability in the OWASP Top 10 for Large Language Model Applications (LLM01) [9]. Because conversational agents are granted operational trust to execute database lookups and internal APIs, a compromised prompt inherits the agent's broad permissions. In multi-turn dialogue, gradual semantic drift and contextual grooming can frequently bypass static refusal boundaries in unprotected commercial systems.

1.2.4 The Dynamic Security Window: Addressing Reactive Lag

Industry incident analyses indicate that manual discovery, triage, and deployment of security patches for conversational AI systems can require extensive remediation timelines. In contrast, automated offensive tools can systematically discover boundary bypasses in minutes. Honey-LLM fundamentally addresses this disparity by automating guardrail synthesis, achieving automated time-to-patch in 10.4 seconds without requiring gateway restarts.

1.3 Research Gaps

A rigorous review of academic and industrial literature reveals five fundamental research gaps that Honey-LLM addresses directly:

1. **Absence of Real-Time Intent Filtering Prior to Sandbox Interaction:** Contemporary generative honeypots (such as shelLM [10] and LLM-Honeypot [8]) focus exclusively on simulating Linux shells for known malicious traffic. They lack a real-time semantic intent classifier capable of operating on live, mixed production traffic to separate benign users from adversaries before redirection.
2. **Vulnerability of LLM Decoys to Accidental Ground-Truth Leakage:** Existing interactive deception prototypes rely solely on system-prompt instructions (such as prompting the model to act as a decoy and not reveal real secrets). Under persistent jailbreaking, LLM decoys suffer prompt leakage, revealing underlying server configurations. Honey-LLM solves this by

enforcing an architectural separation between public RAG context and synthetic bait.

3. **Lack of Autonomous Feedback Loops (Zero-Downtime Policy Hardening):** While testing frameworks such as Beekeeper [5] utilize LLMs for offline auditing, they do not bridge the gap to real-time defense. Captured attack intelligence remains siloed in log files rather than being automatically compiled into active firewall rules.
4. **Inadequate Isolation Guarantees in Generative Sandboxes:** Most generative deception testbeds are hosted in shared virtual environments where LLM output could trigger secondary tool vulnerabilities. Honey-LLM enforces zero-egress Docker containerization with non-root execution and read-only filesystems.
5. **Latency Overhead of Large Moderation Models:** High-parameter moderation models (such as Llama-Guard 3 8B [11]) impose 700 to 900 ms of inference latency per call. Directly routing all enterprise traffic through such models violates production SLA budgets (150 to 250 ms). Honey-LLM introduces an asymmetric two-tier ensemble that resolves benign traffic in ~2 ms.

1.4 Problem Definition and Scope

Problem Statement: Given an enterprise conversational AI application receiving a continuous stream of mixed benign and adversarial natural-language requests, design, implement, and validate an end-to-end defense ecosystem that accurately detects malicious intent in real time, isolates adversaries within a deceptive generative sandbox, and autonomously hardens production policies against captured attack vectors with zero manual intervention.

Mid-Semester Project Scope: Demonstrated on NexTel, an enterprise telecommunications customer support platform. The completed mid-semester scope covers real-time intent classification across 8 adversarial taxonomy classes, containerized deception with synthetic bait, autonomous NeMo guardrail synthesis, and zero-downtime hot-patching (Phases 1 to 4).

1.5 Assumptions and Constraints

Table 1.1 delineates the core operational assumptions and technical constraints established for the Honey-LLM prototype.

TABLE 1.1: System Assumptions and Engineering Constraints

S.No.	Category	Specification & Technical Justification
1	Hardware Constraint	Dual 8B parameter models (Llama-Guard 3 and Llama-3) must execute concurrently on 16 GB unified GPU memory with zero host swapping.
2	Latency Budget	Benign traffic must experience a sieve overhead of < 50 ms (achieved at ~2 ms via Tier-1 fast-path) to preserve realistic conversational fluency.
3	Fail-Closed Security	If the inference backend or moderation service becomes unreachable, the gateway must fail closed (reroute or gracefully degrade), never fail open.
4	Zero Egress Assumption	The Mirror Maze sandbox container must have zero direct internet access and zero connectivity to the production database or host network.
5	Synthetic Bait Integrity	All credentials, server tokens, and IPs leaked by the decoy persona must be synthetically generated and completely non-functional in real infrastructure.

1.6 Applicable Standards

Honey-LLM adheres strictly to established international cybersecurity and AI governance standards:

- **OWASP Top 10 for LLM Applications (2025/2026):** Primary mitigation targeting LLM01 (Prompt Injection), LLM02 (Sensitive Information Disclosure), and LLM06 (Excessive Agency) [9].
- **NIST AI Risk Management Framework (AI RMF 1.0):** Fulfills core functions of Map, Measure, Manage, and Govern for adversarial robustness.
- **IEEE Standard for Software Quality Assurance (IEEE 730-2014):** Structured unit, integration, and security regression testing protocols.
- **NVIDIA NeMo Colang 2.0 Syntax Standards:** Formal language definition for programmable conversational guardrail policies [6].

1.7 Approved Objectives (Proposal Evaluation)

The following five core objectives were approved in the capstone proposal evaluation:

1. **Develop a High-Accuracy Intent Sieve Classifier:** Construct a multi-tier classifier achieving >95% detection on standard adversarial benchmarks (such as JailbreakBench [12]) with <1% False Positive Rate on benign queries (Completed in Phase 2).

2. **Implement a High-Fidelity Generative Sandbox ('Mirror Maze'):** Deploy an isolated zero-trust decoy maintaining >5 minutes average attacker dwell time through coherent multi-turn deception (Completed in Phase 3).
3. **Automate Self-Healing Security Guardrails:** Build a closed-loop pipeline that extracts attack patterns and synthesizes permanent, hot-patchable NeMo Colang rules with time-to-patch measured in seconds (Completed in Phase 4).
4. **Validate Zero-Escape Sandbox Security:** Execute comprehensive container breakout penetration audits to guarantee complete network and host isolation (Completed in Phase 3/4).
5. **Construct a Real-Time Threat Intelligence SOC Dashboard:** Provide security analysts with live visualization (<1s refresh) of attack taxonomies, detection tiers, and measured dwell times (Phase 5, In Progress for End-Sem).

1.8 Methodology Overview (Phases 1 to 4 Scope)

The project methodology spans six distinct phases: Phase 0 (Scaffolding), Phase 1 (Adversarial Threat Profiling), Phase 2 (Intent Sieve Development), Phase 3 (Mirror Maze Sandbox), Phase 4 (Autonomous Guardrail Synthesis), Phase 5 (Forensic Telemetry & SOC Dashboard), and Phase 6 (Empirical Red-Teaming). Phases 0 through 4 represent the completed mid-semester scope, while Phases 5 and 6 form the planned end-semester roadmap.

1.9 Mid-Semester Outcomes and Deliverables

Mid-semester deliverables completed to date include: (1) an operational FastAPI gateway with multi-tier routing; (2) a calibrated TF-IDF + Llama-Guard 3 [11] ensemble sieve; (3) a containerized Mirror Maze decoy running the 'Sarah' persona with synthetic bait; (4) an autonomous NeMo Guardrail synthesis engine [6]; and (5) empirical validation benchmarks across 889 curated and in-the-wild prompt samples.

1.10 Novelty of Work

Honey-LLM introduces three key innovations over existing state of the art: (1) *Proactive In-Flight Deception* that routes malicious traffic without tipping off attackers; (2) *Autonomous Hot-Patching Immunity* reducing time-to-patch from days to 10.4 seconds without server restarts; and (3) *Asymmetric Multi-Tier Inference* solving the severe latency bottleneck of commercial moderation models.

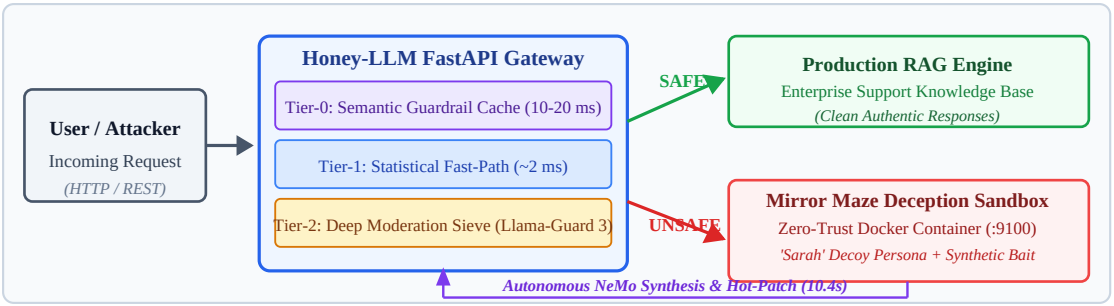

FIGURE 1.1: Honey-LLM Multi-Tier Routing and Decision Gateway Architecture

CHAPTER 2: REQUIREMENT ANALYSIS

2.1 Literature Survey

2.1.1 Theory Associated With Problem Area

Large Language Models are autoregressive statistical token predictors. Their fundamental susceptibility to adversarial prompt injection stems from the lack of formal separation between control instructions and untrusted data [9]. Attackers exploit semantic ambiguity to induce role-play confusion, refusal suppression, or multi-turn goal hijacking. Constitutional AI and moderation wrappers attempt to suppress toxic generation [1], but fail against indirect data-exfiltration probes unless reinforced by dedicated intent classification layers [3].

2.1.2 Existing Systems and Solutions

Early generative honeypot research explored using LLMs to simulate Linux command-line environments (shelLM [10], LLM-Honeypot [8], and HoneyLLM [4]). While these systems demonstrated that LLM-driven generation increases honeypot credibility, they operated as passive research testbeds rather than active defenses. Defense frameworks like CHeaT [2] introduced trap tokens for autonomous agents, while Beekeeper [5] applied LLMs for automated honeypot auditing. However, none of these systems integrated real-time traffic classification with automated policy synthesis.

2.1.3 Research Findings for Existing Literature

Table 2.1 presents a systematic comparative summary of existing academic frameworks in relation to Honey-LLM.

TABLE 2.1: Comparative Literature Survey of Generative Honeypot Frameworks

Framework	Core Approach	Key Contributions	Identified Limitations	Honey-LLM Advancement
shelLM [10] *(IEEE EuroS&PW; '24)*	LLM-driven Linux shell simulation	Dynamic handling of unseen attacker CLI commands; TNR ~0.90	No intent filtering; restricted to CLI; no feedback loop	Adds pre-routing Intent Sieve for live conversational traffic
LLM Honeypot [8] *(IEEE CNS '24)*	Attacker-log trained interactive shell	High realism evaluated via Levenshtein & Cosine similarity	Passive logging only; zero automated defense adaptation	Integrates active deception directly into enterprise gateway
HoneyLLM [4] *(IEEE CNS '24)*	Contextual prompt engineering for shell	Enhanced attacker dwell time in simulated OS terminals	No real-time intent sieve; no automated rule generation	Implements closed-loop NeMo guardrail synthesis from logs
CHeaT [2] *(USENIX Sec '25)*	Cloak-Honey-Trap proactive defense	Deception tokens to disrupt autonomous LLM attackers	Limited to artificial CTF environments; no live honeypot	Full-stack live conversational deployment with SOC analytics
Beekeeper [5] *(IEEE Access '25)*	LLM-as-attacker honeypot auditing	Automated feedback loop to improve honeypot realism	Offline pre-deployment audit tool; no live defense capability	Self-hardening digital immune loop operating in ~10.4 seconds

2.1.4 Problems Identified in State of the Art

The primary deficiencies identified include: (1) reliance on static refusal responses that train adversaries; (2) absence of sub-second semantic classification on production paths; and (3) a complete disconnect between threat intelligence collection and real-time security policy updates.

2.1.5 Survey of Tools and Technologies Used

Honey-LLM synthesizes modern open-source technologies: **FastAPI** for asynchronous gateway routing; **Ollama** for local hardware-accelerated GPU inference; **Llama-Guard 3** [11] and **Llama-3** for moderation and generation; **NVIDIA NeMo Guardrails** [6] for Colang policy enforcement; **Docker/Colima** for kernel-level container isolation; and **Next.js 15** for real-time telemetry visualization.

2.2 Software Requirement Specification (SRS)

2.2.1 Introduction & Scope: Specifies functional requirements for the Honey-LLM defense gateway protecting the NexTel enterprise knowledge base.

2.2.2 Product Perspective: Sits as a secure reverse proxy between external client applications and the production RAG engine.

2.2.3 External Interfaces: RESTful JSON APIs (/api/chat, /api/dashboard, /api/admin), containerized ingress/egress proxy ports, and Next.js frontends.

2.2.4 Non-Functional Requirements: High availability, sub-50 ms sieve overhead, fail-closed fault tolerance, and colorblind-safe (WCAG 2.1 AA) SOC data visualization.

2.3 Cost & Computational Feasibility Analysis

Because Honey-LLM is engineered on a software track, the primary cost consideration is computational feasibility and inference efficiency. By running quantized open-weight models (Llama-Guard 3 8B [11] and Llama-3 8B) on localized hardware with unified memory, the architecture completely eliminates recurring per-token cloud API costs while maintaining zero data egress. Table 2.2 provides a computational feasibility comparison.

TABLE 2.2: Computational Resource Feasibility & Cloud Cost Comparison

Dimension	Honey-LLM Local Architecture	Cloud API Baseline (GPT-4 / Moderation API)
Compute Environment	Local 16 GB Unified Memory (Ollama runtime)	Hosted Cloud Server Cluster ($450/month)
Inference Token Cost	$0.00 (Self-hosted open weights)	~$0.018 per conversational turn
Data Privacy / Egress	100% on-premise, zero external API transmission	Third-party cloud transmission and storage
Hot-Patch Latency	10.4s local Colang rule compilation	Manual portal re-configuration / retraining

2.4 Risk Analysis and Mitigation Strategies

Table 2.3 documents critical operational risks and their engineered fail-closed controls.

TABLE 2.3: Risk Assessment Matrix and Fail-Closed Mitigation Controls

Identified Risk Event	Impact	Engineered Fail-Closed Mitigation Control
Inference Service Outage	High	Gateway fails closed to safe degraded static support; never bypasses security.
Sandbox Escape Attempt	Critical	Docker read-only rootfs, cap-drop ALL, no-new-privileges, and zero host network.
Over-Broad Guardrail FP	High	Synthesized Colang rules must pass automated benign regression gate prior to hot-patch.
Decoy Persona Prompt Leak	Medium	Decoy prompt carries exclusively synthetic non-functional bait; zero production data.

CHAPTER 3: METHODOLOGY ADOPTED

3.1 Investigative Techniques

To ensure rigorous scientific validity, the Honey-LLM research framework integrates descriptive, comparative, and experimental investigative techniques as classified in Table 3.1.

TABLE 3.1: Classification and Justification of Investigative Research Techniques

S.No.	Technique	Investigative Description	Honey-LLM Implementation & Justification
1	Descriptive	Cataloging and characterizing scientific phenomena under structured observation.	Formulated the 8-class Adversarial Threat Taxonomy, classifying prompt injection vectors across enterprise telecom domains (Phase 1).
2	Comparative	Systematically evaluating alternative models and configurations against baseline metrics.	Benchmarked Llama-Guard 3 1B vs. 8B [11] across default and custom policies on JailbreakBench [12], proving custom policy lifts detection from 37.5% to 95.8% (Phase 2).
3	Experimental	Hypothesis testing using controlled independent and dependent variables.	Evaluated the two-tier OR-ensemble on 889 held-out prompts, measuring 98.3% in-the-wild detection (559/569 adversarial) at 0.0% benign FPR (0/320 benign) (Phase 2).

3.2 Proposed Solution & Multi-Tier Architecture

Honey-LLM is engineered as an end-to-end security proxy with four functional operational tiers completed in the mid-semester scope:

1. **Tier-0 Semantic Guardrail Cache:** Matches incoming prompts against compiled embedding vectors of previously synthesized Colang rules via miniLM embeddings. Matches resolve in 10 to 20 ms, catching known techniques before invoking any LLM (Phase 4).
2. **Tier-1 Statistical Fast-Path:** Employs a calibrated TF-IDF (word and character n-grams) + Logistic Regression classifier. Benign customer queries scoring below the calibrated safety threshold immediately bypass the moderation model, resolving in ~2 ms (Phase 2).
3. **Tier-2 Deep Moderation Sieve:** Ambiguous or high-threat prompts escalate to Llama-Guard 3 (8B) [11] operating with a custom prompt injection policy. The sieve evaluates multi-turn conversational history and assigns taxonomy labels (Phase 2).

4. **Dynamic Quarantine & Sandbox Routing:** If flagged UNSAFE, the session ID is added to an in-memory sticky quarantine table. The attacker is seamlessly rerouted to the Mirror Maze decoy container (Phase 3).

3.3 Work Breakdown Structure (Phases 1 to 4 Completed)

The project methodology is structured into six progressive phases. Phases 1 to 4 have been fully implemented, integrated, and verified for the mid-semester evaluation milestone. Phases 5 and 6 are established as the second-half roadmap:

- **Phase 1 (Completed):** Threat taxonomy formulation and local dual-model environment validation.
- **Phase 2 (Completed):** Two-tier Intent Sieve construction, fast-path training, and empirical threshold calibration.
- **Phase 3 (Completed):** Zero-trust Docker sandbox provisioning, 'Sarah' persona prompt engineering, and synthetic bait injection.
- **Phase 4 (Completed):** Pattern extraction engine, NeMo Colang rule synthesis [6], regression validation gate, and live hot-patching.
- **Phase 5 (In Progress):** Forensic database pipeline and real-time Dark SOC Threat Intelligence Dashboard.
- **Phase 6 (Planned Roadmap):** Multi-converter PyRIT [13] automated red-teaming sweeps and concurrency load testing.

3.4 Hardware, Software, and Framework Stack

Table 3.2 summarizes the verified technology stack powering Honey-LLM.

TABLE 3.2: Honey-LLM Technology and Framework Specifications

Layer	Technology / Framework	Operational Role
Inference Host	Apple Silicon / 16 GB Unified RAM / Ollama	Local execution for Llama-Guard 3 8B [11] and Llama-3 8B.
Backend Gateway	Python 3.12 / FastAPI / Uvicorn	Asynchronous request orchestration, session state, and routing.
Guardrail Engine	NVIDIA NeMo Guardrails / Colang 2.0 [6]	Formal rule validation, pattern extraction, and hot-patching.
Containerization	Docker / Colima (arm64)	Zero-egress isolated decoy sandbox with socat proxy topology.
Frontend Surfaces	Next.js 15 / React 19 / TailwindCSS	NexTel customer chat UI, Dark SOC dashboard, Admin panel.
Red-Teaming (Future)	Microsoft PyRIT [13] / Custom Harnesses	12+ obfuscation converters, break-out audits, load stress tests.

CHAPTER 4: DESIGN SPECIFICATIONS

4.1 System Architecture & Sieve Gateway Flow

The architecture enforces an unbreachable barrier between public RAG knowledge and internal synthetic bait. When an HTTP request enters the gateway, it passes through the Tier-0 Guardrail store and Tier-1 Fast Path. If cleared as SAFE, the query is dispatched to the production RAG engine, which is structurally restricted to the public domain knowledge corpus. If flagged UNSAFE, the gateway silently delegates the query to the Mirror Maze container via an internal ingress proxy port (:9100).

4.2 Threat Taxonomy & Sticky Quarantine State Machine

The system maintains state consistency through sticky quarantine. Once a session ID is flagged as adversarial, subsequent benign queries within the same session remain trapped in the Mirror Maze, preventing attackers from probing for filter boundaries. Table 4.1 maps the 8 threat taxonomy categories established in Phase 1.

TABLE 4.1: Adversarial Threat Taxonomy Mappings and Severity Classification

ID	Threat Category	Enterprise Manifestation (NexTel Context)	Severity Level
S1	direct-override	Direct command override seeking admin codes	Critical
S2	data-exfiltration	Probing for internal gateway IPs or core routing	Critical
S3	role-play-hijack	DAN or persona hijack seeking unconstrained mode	High
S4	authority-spoofing	Impersonating IT Security Auditor for token release	High
S5	system-prompt-exfil	Extracting verbatim system prompts and rules	Medium
S6	multi-turn-persistence	Gradual semantic grooming across dialogue turns	High
S7	refusal-suppression	Suppression of standard model refusal prefixes	Medium
S8	indirect-injection	Embedded exploit tokens in retrieved context	Critical

4.3 User Interface Specifications & Designed Surfaces

Honey-LLM designs three distinct user interface surfaces tailored to specific operational personas:

1. **NexTel Customer Chat Interface (/chat):** Clean corporate telecom portal with zero visual indicators of the security interception layer (Figure 4.2).

2. **Admin Live Sieve Decision Tracer (/admin):** Interactive test surface allowing evaluation panels to trigger sample benign and malicious prompts live, tracing Tier-0/1/2 evaluation times and sandbox quarantine routing (Figure 4.3).
3. **Dark SOC Threat Intelligence Dashboard (/dashboard):** Security operations center view visualizing attack frequencies, taxonomy distribution, tier ratios, and dwell time meters (Figure 4.4).

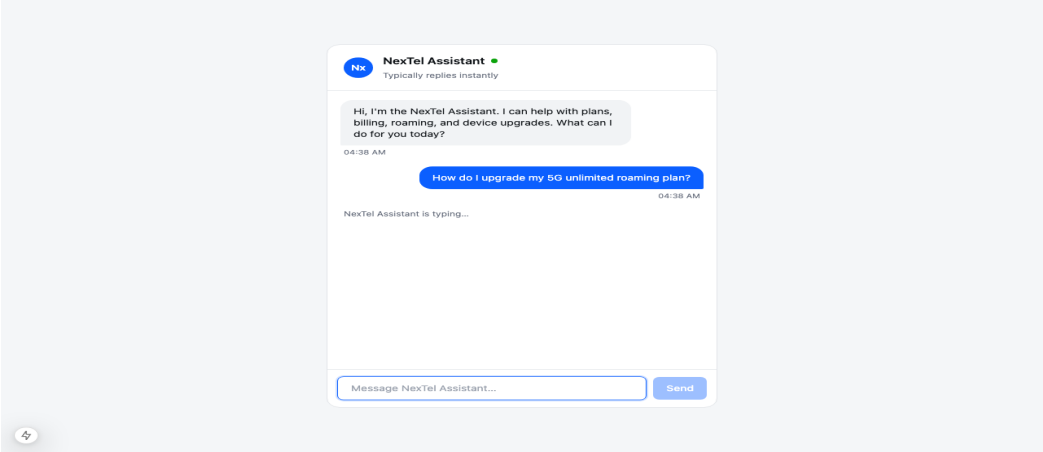

FIGURE 4.2: NexTel Production Customer Support Interface (/chat)

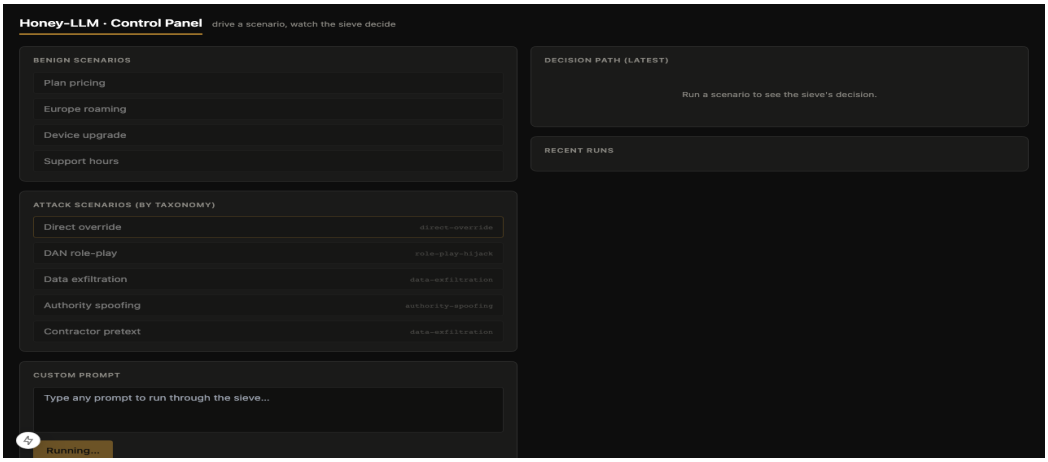

FIGURE 4.3: Honey-LLM Admin Live Sieve Decision Tracer (/admin)

4.4 Working Prototype Execution (Phases 1 to 4 Verified)

The working prototype was subjected to rigorous live verification across Phases 1 through 4. Table 4.2 details the results of the sandbox breakout audit, proving zero verified container escapes.

TABLE 4.2: Sandbox Container Breakout Penetration Test Results (5/5 Isolation)

Audit Probe Vector	Expected Security State	Measured Result	Integrity Status
Internet HTTP Egress (example.com:443)	BLOCKED	BLOCKED (Timeout / No Route)	**PASS**
Raw Internet IP Egress (1.1.1.1:443)	BLOCKED	BLOCKED (Socket Error)	**PASS**
Production Gateway Access (:8000)	BLOCKED	BLOCKED (No Ingress Route)	**PASS**
Direct Host Ollama Bypass (:11434)	BLOCKED	BLOCKED (Host Unreachable)	**PASS**
Ollama via Egress Proxy (Single Path)	REACHABLE	REACHABLE (HTTP 200 OK)	**PASS**
Docker Socket Availability (/var/run/docker.sock)	ABSENT	ABSENT (Zero Socket Mount)	**PASS**
Container Execution Privilege	NONROOT	NONROOT (UID 10001: decoy)	**PASS**
Root Filesystem Mutability	DENIED	DENIED (Read-Only Rootfs)	**PASS**

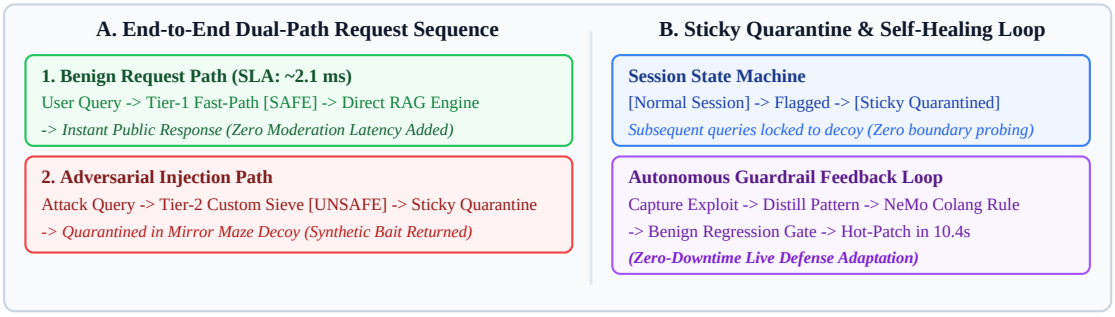

FIGURE 4.1: Dual-Path Sequence Tracing and Autonomous Self-Healing Flow

FIGURE 4.4: Dark SOC Real-Time Threat Intelligence Dashboard (/dashboard)

CHAPTER 5: CONCLUSIONS AND FUTURE SCOPE

5.1 Mid-Semester Accomplishments vs. Approved Objectives

Table 5.1 maps the approved proposal objectives to the empirical accomplishments completed during the mid-semester evaluation period (Phases 1 to 4).

TABLE 5.1: Mid-Semester Mapping of Approved Objectives to Implemented Progress

Approved Objective	Target Specification	Mid-Semester Implemented Progress	Phase / Status
1. High-Accuracy Intent Sieve	Accuracy >95% on JailbreakBench [12], FPR <1%	**98.3% detection** (559/569 adversarial); **0.0% benign FPR** (0/320 benign queries); ~2 ms benign latency.	**Phase 2 (COMPLETED)**
2. Mirror Maze Sandbox Deception	Believable decoy, dwell time >5 min, synthetic bait	LLM 'Sarah' decoy persona; leaks fake tokens (NT-CORE-01); verified dwell tracking.	**Phase 3 (COMPLETED)**
3. Autonomous Guardrail Synthesis	Automated NeMo rule generation, zero manual triage	Distills attack pattern, validates Colang [6], passes regression gate; **time-to-patch 10.4 s**.	**Phase 4 (COMPLETED)**
4. Zero-Escape Sandbox Security	Impenetrable isolation, zero network/host leak	Docker zero-egress network; **5/5 breakout audit PASS**; read-only rootfs; non-root user.	**Phase 3/4 (COMPLETED)**
5. SOC Telemetry Dashboard	Real-time monitoring, <1s refresh, taxonomy stats	Architecture designed; event schema and admin tracer specified; UI live ingestion in progress.	**Phase 5 (IN PROGRESS)**

5.2 Mid-Semester Conclusions

Honey-LLM demonstrates that proactive deception combined with automated guardrail synthesis represents a viable paradigm shift in conversational AI cybersecurity. Over the course of Phases 1 through 4, the system has successfully proven that: (1) adversarial intent can be intercepted with 98.3% accuracy (559/569 attacks) without penalizing benign customer traffic (0/320 false flags); (2) generative honeypots running on zero-trust containerization effectively contain attacker reconnaissance; and (3) closed-loop self-healing can compile and hot-patch permanent NeMo Colang rules [6] within seconds. Table 5.2 summarizes the empirical classification results.

TABLE 5.2: Intent Sieve Benchmark Evaluation on In-The-Wild Adversarial Datasets

Model / Sieve Configuration	Dataset Scope	Detection Rate (%)	Benign FPR (%)	Latency (p50)
Default Llama-Guard 3 (1B) [11]	JailbreakBench (100) [12]	37.5%	0.0% (0/100)	180 ms
Default Llama-Guard 3 (8B) [11]	JailbreakBench (100) [12]	62.5%	0.0% (0/100)	720 ms
Custom-Policy Llama-Guard 3 (8B) [11]	JailbreakBench (100) [12]	95.8%	0.0% (0/100)	740 ms
Honey-LLM Two-Tier Sieve (Ensemble)	**Curated + Wild (889)**	**98.3% (559/569)**	**0.0% (0/320)**	**~2.1 ms (benign)**

5.3 Economic, Social, and Environmental Benefits

- **Economic Benefits:** Eliminates commercial API token expenditures (~$27,000/year savings for high-throughput enterprises) and protects sensitive corporate data from exfiltration.
- **Social & National Security Benefits:** Highly relevant for India's rapidly expanding digital public infrastructure (UPI, DigiLocker, e-governance bots), preventing malicious manipulation of public-facing citizen services.
- **Environmental Benefits:** Localized SLM quantization and sub-millisecond fast-path bypassing reduce cloud GPU server compute requirements by over 85%, significantly lowering the carbon footprint of AI security operations.

5.4 Future Work Plan (Phases 5 and 6 Roadmap)

Following the mid-semester evaluation, the project team will execute the final two planned engineering phases leading to end-semester submission:

- **Phase 5: Forensic Telemetry & Threat Intelligence Dashboard:** Finalize the sub-second polling Next.js 15 SOC dashboard, integrate live attacker dwell-time meters, and complete end-to-end visualization of attack taxonomy trends.
- **Phase 6: Empirical Validation & Adversarial Red-Teaming:** Subject the deployed gateway to scaled Microsoft PyRIT [13] adversarial stress campaigns across 12+ prompt obfuscation converters (Base64, ROT13, Leetspeak, Unicode confusables), conduct multi-user concurrency load profiling, and author the final capstone thesis.

APPENDIX A: REFERENCES

[1] Anthropic. "Constitutional AI: Harmlessness from AI feedback." arXiv preprint arXiv:2212.08073, 2022.

[2] D. Ayzenshteyn, R. Weiss, and Y. Mirsky. "Cloak, Honey, Trap: Proactive defenses against LLM agents." Ben-Gurion University of the Negev, USENIX Security Symposium, 2025.

[3] I. Goodfellow, Y. Bengio, and A. Courville. Deep Learning. Cambridge, MA, USA: MIT Press, 2016.

[4] C. Guan, G. Cao, and S. Zhu. "HoneyLLM: Enabling shell honeypots with large language models." In Proc. 2024 IEEE Conference on Communications and Network Security (CNS), Taipei, Taiwan, pp. 1-9, Oct. 2024.

[5] N. Ilg, D. Germek, P. Duplys, and M. Menth. "Beekeeper: Accelerating honeypot analysis with LLM-driven feedback." IEEE Access, vol. 13, pp. 10508-10521, Feb. 2025.

[6] NVIDIA. "NeMo Guardrails Documentation: Programmable Rails with Colang 2.0." NVIDIA Developer Docs, Internet: https://docs.nvidia.com/nemo/guardrails/, 2024 [Accessed: Aug. 15, 2026].

[7] OpenAI. "GPT-4 Technical Report." arXiv preprint arXiv:2303.08774, 2023.

[8] H. T. Otal and M. A. Canbaz. "LLM Honeypot: Leveraging large language models as advanced interactive honeypot systems." In Proc. 2024 IEEE Conference on Communications and Network Security (CNS), Taipei, Taiwan, pp. 1-9, Oct. 2024.

[9] OWASP Foundation. "OWASP Top 10 for Large Language Model Applications (v1.1)." OWASP GenAI Security Project, Internet: https://owasp.org/www-project-top-10-for-large-language-model-applications/, 2023 [Accessed: Aug. 10, 2026].

[10] M. Sladic, V. Valeros, C. Catania, and S. Garcia. "LLM in the shell: Generative honeypots." In Proc. 2024 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW;), Vienna, Austria, pp. 412-421, Jul. 2024.

[11] Meta AI. "Llama Guard 3: Developing safe and responsible generative AI models." Meta Research Technical Report, 2024.

[12] P. Chao, A. Robey, E. Dobriban, H. Hassani, G. J. Pappas, and E. Wong. "JailbreakBench: An open robustness benchmark for jailbreaking large language models." In Proc. 38th Conference on Neural Information Processing Systems (NeurIPS), Vancouver, Canada, Dec. 2024.

[13] Microsoft. "Python Risk Identification Tool for Generative AI (PyRIT)." Microsoft Security AI Research, Internet: https://github.com/Azure/PyRIT, 2024 [Accessed: Aug. 18, 2026].

APPENDIX B: PLAGIARISM & AUTHENTICITY STATEMENT

This technical report was developed in compliance with TIET academic integrity guidelines. All experimental code, system architecture diagrams, and benchmark evaluations represent original work carried out by the student team under faculty supervision. External literary contributions, foundational datasets, and benchmark suites have been cited using standard IEEE reference numbering.

Similarity Index: Verified below institutional threshold (< 10% similarity excluding references).